SIX DEGREES

PHILLIP S, KALEN HAZLETT, MORGAN QUACKENBUSH

OBJECTIVE

- Create a program that finds minimum separation between users based on mutual connection
- Visualize the graph of users with their connections

PROJECT FLOW

- Rust prompts the user to enter in two usernames
- Rust then sends a username to the mutuals checker via command line.
- Python parses database file and returns intersection of followers and following
- Process repeats until graph is mapped out, then BFS finds the shortest path.
- Data is sent to the visualizer, which plots the formed graph.

TWO LANGUAGES?

- Originally Going to Web Scrape
  - Twitter API
- Good Data Science Libraries in Python
- Rust is emerging in critical systems and is compiled --> faster
  - Has 'memory safety'
- Twitter API is expensive, and we are broke, so we created our own database

DATABASE

- Collection of tables (followers and following) that can search itself
- Contains an index that organizes the tables into B-Trees, making searching much faster (O(log n))
- Without the index, it would be no different than a line-by-line search

MUTUAL SORT

- Receives an account that needs its mutuals found.
- Runs an SQL Query that looks in the followers table and following table and collects the rows relevant to the user.
- Extracts the data (the follower or followee) and discards the user.
- Puts that extracted data into sets.
- Finds the intersection of the two sets and sends it back through command line

RUST

- Main data structure of the project
  - Has the graph represented as an adjacency list
  - Computes BFS traversal to find shortest/only path
    - Trends towards O(V+E)

GRAPH

```rust
pub struct Graph<N, W = ()>{
    nodes: Vec<N>, //creates the nodes to
    adj: Vec<Vec<(NodeId,W)>> //creating the list (neigbor, weight)
}

//methods to work on the structure above. Like Classes in Python or objects in C++
impl<N, W> Graph<N, W>{
    //public function: create a new node
    pub fn new() -> Self{
        //create a new node using the vector library imported at top Vectors are lists that can change size safely
        Self { nodes: Vec::new(), adj: Vec::new()}
    }

    //function to add a new node to the graph (&mut means a parameter is going to be a mutable variable)
    pub fn add_node(&mut self, data: N) -> NodeId{
        let id = self.nodes.len(); //creates a variable of usize

        self.nodes.push(data);
        self.adj.push(Vec::new());

        //returns the id
        id
    }
```

COMMUNICATION

```rust
//spawn the python script as a sub process
let mut child: Child = Command::new(program: "python3") Command
    .arg("-u") &mut Command
    .arg("py_script_2.py") &mut Command//change for actual script name. Use .env file if needed/want
    .stdin(cfg: Stdio::piped()) &mut Command
    .stdout(cfg: Stdio::piped()) &mut Command
    .spawn()?;

//creating variables for data
let mut py_stdin: ChildStdin = child.stdin.take().expect(msg: "failed to open python stdin");
let py_stdout: ChildStdout = child.stdout.take().expect(msg: "failed to open python stdout");
let mut reader: BufReader<ChildStdout> = BufReader::new(inner: py_stdout);

//create some variables for traversing later
let mut g: Graph<String, f64> = Graph::new();
let mut index: HashMap<String, NodeId> = HashMap::new();

//could have read in from a file or gotten input, but why? this works for the sample data we have
let usernames: Vec<&str> = vec!["a", "b", "c", "d", "e", "f", "g", "h", "i", "j",
                                "k", "l", "m", "n", "o", "p", "q", "r",
                                "s", "t", "u", "v", "w", "x", "y", "z"];
//loop through the list of usernames, parsing each one
for u: &&str in &usernames {
    //normalize the input
    let username: String = canon(u);
    // send username to the python script
    writeln!(py_stdin, "{}", username)?;
    py_stdin.flush()?; //flush the buffer

    let mut line: String = String::new();
    let n: usize = reader.read_line(buf: &mut line)?; //read the python line
```

VISUALIZATION

- Receives data from Rust code in json
  - Graph Information
  - Traversal Information
- Plots Each user
- Creates edges in between each user
- Creates the path of traversal

DEMONSTRATION

CHALLENGES

- Conceptual Understanding of the Project
- Using Two languages (Using Rust)
- Finding/creating a data set that worked for our purposes
- Debugging
  - Json parsing

POTENTIAL ENHANCEMENTS

- Live separation checker
- Larger database
- Use actual API (looking for sponsors)

RESOURCES AND AI STATEMENT

- Multiple online articles
- OpenAI ChatGPT model 5.0 for debugging and language translation

QUESTIONS?